

WORKLOAD-SHAPED LLM SERVING

SmolLM2-360M-Instruct on CPU and RTX 6000 Ada

DSAA4012
FINAL PROJECT

Hanxiao
Haiyang Peng
23 July 2026

Abstract

We characterize how batch size and context length reshape the time-to-first-token (TTFT), time-per-output-token (TPOT), throughput, and memory frontier of SmolLM2-360M-Instruct. A deterministic manual decode loop separates prefill from steady decode on one AMD EPYC 9654 socket and one RTX 6000 Ada GPU. We then evaluate scaled-dot-product attention (SDPA), KV caching, and dynamic W8A8 quantization. The main result is conditional: no intervention is universally optimal. SDPA is increasingly valuable at long context; cache bookkeeping can slightly hurt the smallest GPU loads but avoiding the cache becomes 4.22× slower at context 2048, batch 8; CPU W8A8 improves throughput by 1.27–2.49× yet increases peak RSS and severely degrades perplexity and ARC-Easy accuracy. Correcting the benchmark to retain only final-position logits also removes artificial OOM boundaries, showing that measurement semantics are part of the systems result.

1. Problem and system overview

Serving studies often reduce optimization to a single tokens-per-second number. That hides the trade-off between prefill latency, decode latency, batch efficiency, and memory capacity. Our question is therefore a frontier question: as context and batch change, which bottleneck appears, and which implementation choice moves the frontier without unacceptable memory or quality cost?

The model is HuggingFaceTB/SmolLM2-360M-Instruct: a 361.8M-parameter decoder-only Llama-family model with 32 layers, hidden size 960, intermediate size 2560, 15 query heads, 5 KV heads, grouped-query attention, and an 8192-token maximum context [1]. The compact model is deliberate: it can underutilize a large GPU at small batch and exposes overhead, bandwidth, attention, and matrix-throughput transitions within an accessible experiment budget.

2. Experimental design

Prefill consumes a prepared prompt and ends when the first generated token becomes available. Decode then generates 64 tokens synchronously with greedy selection. Model-only TTFT excludes tokenization; TPOT is the elapsed time after the first token divided by the remaining 63 steps; aggregate TPS counts all output tokens across the batch. CUDA is synchronized at token boundaries. Baseline grids span contexts {128, 512, 2048, 4096}, CPU batches {1, 2, 4, 8, 16}, and GPU batches {1, 2, 4, 8, 16, 32, 64, 128}. Each broad-grid point has ten repetitions; two GPU headline points have 100 repetitions in alternating blocks.

The batching knee is the first tested increase with <10% TPS gain while p95 TTFT or TPOT still rises. Comparisons remain within one hardware line; CPU and GPU absolute values are not treated as a fairness comparison. Quality uses a frozen 12,846-token WikiText-2 slice plus full ARC-Easy and HellaSwag evaluations.

3. Implementation

3.1 Benchmark harness and correctness boundary

The repository provides YAML-composed configurations, a deterministic prompt generator, a manual prefill/decode loop, token-level timing, process/device memory measurement, JSONL schema validation, resume-safe manifests, aggregation, plotting, and quality scripts. Each result records the merged configuration, software versions, model artifact hashes, clean Git revision, and invocation history.

A critical correction changed every model forward from full-sequence logits [B,L,V] to final-position logits [B,1,V] using logits_to_keep=1. The legacy records remain labeled full_logits_prefill and are never combined with schema-v3 aggregates. Under the corrected semantics, former GPU OOM points at 2048/128 and 4096/64 succeed; 4096/128 also succeeds at 31.95 GiB peak allocated memory. This confirms that the old boundary came from materializing unused vocabulary logits, not serving-representative state.

3.2 Platforms and controlled runtime

CPU line	One AMD EPYC 9654 NUMA/socket side; logical CPUs 0–95; 96 intra-op and 1 inter-op thread; FP32.
GPU line	GPU0 of an eight-GPU host; NVIDIA RTX 6000 Ada 48 GB; BF16; other devices unused.
Attention	PyTorch eager vs SDPA. CUDA profiling confirms pytorch_flash::flash_fwd_kernel; eager exposes bmm/softmax.
Caching	Dynamic KV cache enabled vs disabled at representative context/batch points.
Quantization	CPU eager dynamic W8A8 on 224 Transformer Linear modules; lm_head retained in FP32.

3.3 Reproducibility and validation

The corrected baseline contains 200 CPU and 320 GPU observations. Operator experiments cover five representative points per hardware line with ten repetitions per implementation. Cache experiments use repeated points except for one explicitly labeled CPU 2048/8 boundary. The final repository passes 22 model-free unit tests and Ruff. Hardware, environment, test, lint, telemetry, profiler, raw, processed, and plot artifacts are committed together.

- Tail claims use the 100-repetition files; broad-grid p99 values from ten repeats are descriptive only.
- The 100-repetition 128/1 GPU point has p99 TTFT 44.67 ms; 4096/128 has p99 TTFT 6.923 s.
- Telemetry spans 29–70 °C without progressive failure or a memory leak.

3.4 Interpretation discipline

A faster quantized run is not attributed to INT8 without kernel evidence; a slower path is retained and explained; and memory claims distinguish allocated/reserved GPU memory from fresh-process CPU RSS. SDPA's fused behavior is consistent with IO-aware attention work [2] and PyTorch's backend selection [4], but our claims are limited to the kernels actually observed.

4. Evaluation

4.1 Baseline surface

CPU median TPS peaks at 146.96, 119.91, 50.95, and 25.01 for contexts 128, 512, 2048, and 4096. The frozen rule finds knees at 2048/16 and 4096/8, with none through batch 16 for shorter contexts. GPU median TPS reaches 2973.59, 2947.87, 2700.83, and 1437.52 at batch 128. Only context 4096 reaches a knee at batch 128; the remaining curves still scale within the tested range.

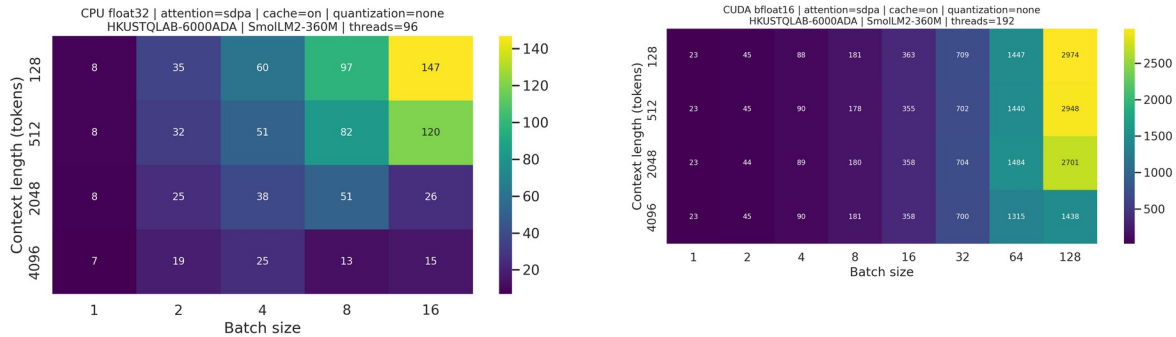


Figure 1. Corrected median decode TPS surfaces. CPU (left) peaks earlier at long context; GPU (right) continues scaling to batch 128.

4.2 Attention and KV cache

Eager/SDPA median TTFT ratios rise from 1.00 to 6.23 on CPU and from 1.40 to 9.34 on GPU across the selected workload points. GPU TPOT stays near a 1.30× eager penalty, whereas CPU TPOT grows to 4.09× at 4096/4. The profiler distinguishes explicit eager bmm/softmax from SDPA's fused FlashAttention kernel, supporting a causal operator-level explanation rather than a label-only comparison.

Caching shows a crossover. On CPU, cache-off TPOT is 1.85–17.62× cache-on at five repeated points. A single 2048/8 boundary reaches 88.99× and takes 891.7 s; we do not call it a percentile. On GPU, cache-off is 1–5% faster at five smaller loads, then 4.22× slower at 2048/8. For this 360M model, the bookkeeping cost can exceed recomputation before the sequence/batch product is large enough. This also illustrates why production systems invest in explicit KV-memory management [3], even though our local synchronous harness does not implement request scheduling or PagedAttention.

4.3 Quantization, memory, and quality

Intervention	Measured effect	Interpretation
SDPA vs eager	TTFT ratio at 4096/4: 6.23× CPU; 9.34× GPU	Clear long-context win; fused CUDA kernel verified.
KV cache	GPU off/on TPOT: 0.95–0.99× at five small loads; 4.22× at 2048/8	Crossover: bookkeeping can exceed recomputation for a small model.
CPU W8A8	TPS 1.27–2.49×; peak RSS 1.12–1.66×	Speed optimization on this eager path, not a memory reduction.
W8A8 quality	PPL 11.27→44.16; ARC-Easy 56.44%→42.21%	This recipe is not quality-preserving.

4. Evaluation (continued)

CPU W8A8 increases median TPS by 1.27–2.49×, consistent with reduced weight traffic and available integer matrix kernels, but peak RSS is 1.12–1.66× native. The tested eager dynamic path is therefore a speed optimization, not a resident-memory optimization. Quality cost is substantial: WikiText-2 perplexity rises from 11.27 to 44.16, while full ARC-Easy accuracy falls from 56.44% to 42.21% (normalized: 49.12% to 38.97%). The native HellaSwag result is 42.83% accuracy, or 56.88% with standard length normalization (10,042 examples; ≈0.49-point standard error).

The GPU TorchAO probe is retained as a negative diagnostic. Profiling confirms 225 `aten::_int_mm` calls and 224 Ampere INT8 GEMMs, but also 9,025 device-to-host scalar transfers during one 128/1 prefill. Throughput is only about 0.21 TPS, so we make no full-grid GPU W8A8 speed claim. The result is useful because it separates functional INT8 execution from an efficient end-to-end quantized serving path.

4.4 Synthesis and limitations

The measured selection rule is workload-shaped. Short, small GPU loads are overhead-sensitive and may not benefit from cache state. Larger batches increase utilization until context-driven latency and activation/KV memory tighten the frontier. Long-context prefill strongly favors SDPA. Long, batched decode favors caching. CPU W8A8 is attractive only when its speedup justifies higher RSS and the measured quality loss is acceptable—or when a better calibration/quantization recipe restores accuracy.

These are synchronous, model-only measurements. They exclude tokenization, networking, continuous batching, admission control, request queues, prefix sharing, and multi-tenant effects. The study uses one small model, one CPU platform, one GPU model, and one main framework. Future work should repeat the region map with a production scheduler, larger models, static/compiled cache paths, and alternative quantization recipes. CPU and GPU are separate hardware lines; absolute numbers are not a price/performance or fairness comparison.

5. Contribution statement

Hanxiao led the benchmark framework, reproducibility controls, CPU experiments, and data correction/validation. Haiyang Peng led GPU execution, operator/cache/quantization studies, and quality evaluation. Both authors jointly designed the experiment, interpreted results, reviewed artifacts, and prepared the report and presentation.

6. AI-use acknowledgment

OpenAI Codex was used for repository planning, implementation and debugging assistance, consistency audits, and drafting/editing the presentation and report. The authors executed the experiments, reviewed generated changes, verified raw and processed results against manifests and profiler evidence, reran tests, and remain responsible for all claims. AI-generated text or code was not treated as experimental evidence.

Bottom line. Serving optimization should begin with the workload region, then verify the executed kernel, memory effect, and quality cost.

References

- [1] L. Ben Allal et al., “SmolLM2: When Smol Goes Big—Data-Centric Training of a Small Language Model,” arXiv:2502.02737, 2025. [arXiv record](#)
- [2] T. Dao et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” arXiv:2205.14135, 2022. [arXiv record](#)
- [3] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” arXiv:2309.06180, 2023. [arXiv record](#)
- [4] PyTorch, “torch.nn.functional.scaled_dot_product_attention,” documentation. [PyTorch documentation](#)

Artifact availability

The repository contains source code, pinned configurations, raw JSONL records, manifests, processed CSVs, figures, validation logs, hardware metadata, tests, and exact reproduction commands. Submission-grade performance files contain “_last_logits” and use schema version 3; legacy full-logit files are retained only for provenance.